

② TCP/IP参考模型

4) APPNET最终采用TCP和IP为主要协议.

(2) 分层:

应用层, — DNS, HTTP, FTP, SMTP, -

传输层 - TCP, UDP

互联网层。 — IPv4, IPv6

网络接口层 一 描述了为满足无连接的网络层需求,链路必备功能

13) 特点: 端对端原则, 聪明终端 & 简单网络, 由端系统负责丢失恢复等。

网络简单,从而大大提升了可扩展性

IP分组交换特点: 可在各种底层物理网络上运行 (IP over everything)

可支持各类上层应用 (Everything over ZP)

③ OSI模型 vs TCP/IP模型

(1) 7层 vs 4层 (基于对应关系: 网络接口、物理层、应用层、表示层、会话层)

(2) 通用性 vs 实用性 (OSI 明确} 服务、协议接口等概念, TCP/IP 仅为对已有协议的描述)

1) 网络层能支持无连接和面向连接通信 VS 网络层仅支持无连接通信

(14) 缺点比较

OSI: 从未被真正实现、技术实现糟糕、以及非技术因素。

TCP/IP: 核心概念未体现、不具通用性、混淆端与层的设计、模型欠缺完整性

④ 本课程分层：
应用层
传输层
网络层
数据链路层
物理层

⑤ 度量单位：略

⑤ 度量单位：略



第二章 物理层

1. 物理层基本概念

① 功能

位置: 最低层

功能: 如何在连接各计算机的传输媒体上传输数据比特流

作用: 尽可能地屏蔽掉不同传输媒体和通信手段的差异

② 接口特性

(1) 物理层协议是 DTE (数据终端设备) 和 DCE (数据电路终端设备) 间的约定

规定两者之间的接口特性

↓
数据处理、转发; 数据源点/终点

↓
信号变换、编码; 建立、保持、释放数据链路

(1) 机械特性

定义接线器的形状和尺寸, 引线数目和排列, 固定和锁定装置等

(2) 电气特性

规定了 DTE/DCE 间多条信号线的电气连接、电路特性

· 如发送信号电平、发送/接收器的输出阻抗、平衡特性等

· 发送/接收器的电路特性、负载要求、传输速率、连接距离等

(3) 功能特性

描述接口执行的功能, 定义接线器的每一引脚的作用

(4) 过程特性

指明对于不同功能的各种可能事件的出现顺序

③ 常用标准

广播通信线路: 一条公共通信线路连接多个结点

其物理层标准 传统以太网 IEEE 802.3: 10BASE-T 等



(数据率)

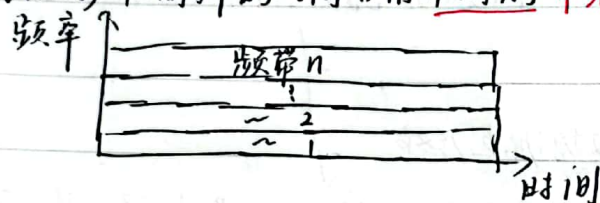
10BASE-T $\rightarrow$ 10: 传输速率 10Mbps, base: 基带信号, T: 非屏蔽双绞线
还有 快速/千兆/万兆以太网, 无线局域网

2. 多路复用技术

⑩ 复用技术的目的: 允许用户使用一个共享信道来通信, 避免相互干扰, 以降低成本, 提高效率

(1) 频分复用(FDM)

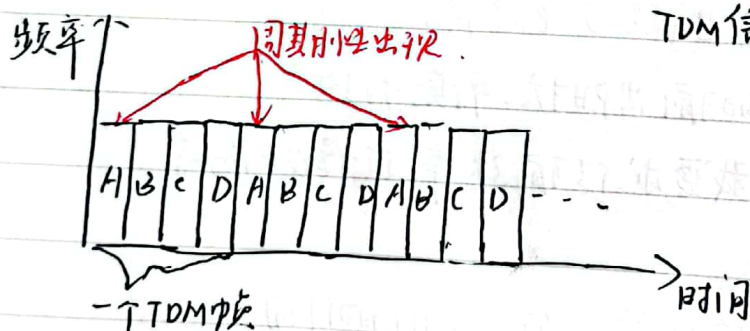
指用户在同样的时间占用不同的带宽资源 (频率带宽)



(2) 时分复用(TDM)

⑪ 将时间划分为一段段等长的时分复用帧

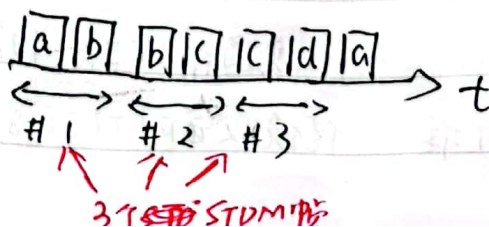
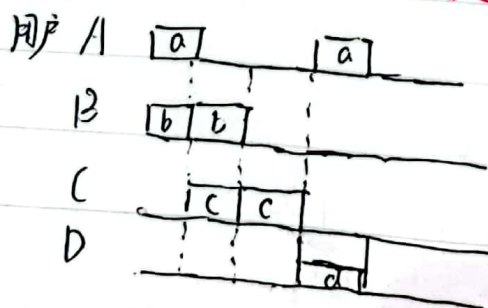
所有用户在不同的时间占用同样的频率带宽



⑫ 不足: 用户暂时无数据发送, 分配给该用户的时隙浪费

⑬ 改进: 统计时分复用(STDM)

指动态地按需分配共用信道的时隙



扫描全能王 创建

13) ~~波分复用~~ 波分复用(WDM)

利用多个激光器在单条光纤上同时发送多束不同波长激光的技术。

(就是光的波分复用, 一根光纤同时传输多个光载波信号。)

14) 码分复用(CDMA)

(1) 码分多址是指利用码序列相关性实现的多址通信, 靠不同地址码区分的地址。

(2) 每一个比特时间划分为 m 个时间间隔, 称为码片。

每个站被指派唯一的一个 m bit 码片序列

例: 5站 8 bit 码片序列是 00011011

发送比特 1 时, 序列不变: 00011011

0 时, 取反码: 11100100

(3) 不同站码片序列各不相同, 且必须相互正交 (规格化内积等于 0)

定理: 任一码片向量与自己的规格化内积为 1, 与自己反码内积为 -1

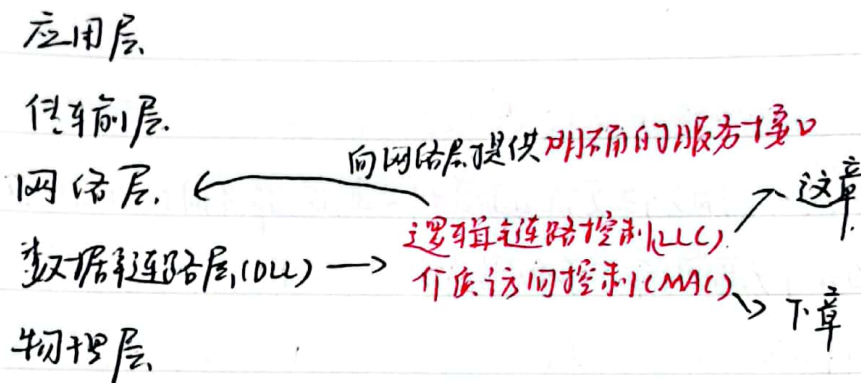
例: P41



第三章 数据链路层基础

一、数据链路层的设计问题

(1) 在协议栈中位置



(2) 功能

① 成帧

将比特流划分成“帧” → 检测、纠正物理层比特传输出现的错误

② 差错控制

处理传输中出现的差错 (位错误、丢失...)

③ 流量控制

使发送方发送速率 ≤ 接收方处理速率

二 差错检测和纠正

(1) 通常, 增加冗余信息 (校验信息)

so, 尽量减少冗余信息量又能检错。

How? — 检错码/纠错码

(1) 检错码

① 奇偶校验

增加一位校验位, 可检查奇数位错误 (两位出错奇偶性不变)



② 校验和 (TCP/IP 体系中主要采用的校验方法) → 校验和
 发送的数据进行 16 位 = 进制补码求和, 结果取反同原数据一同发送。
 接收方同样求和, 多加一个校验和一起求, 求得全 1 则正确, 否则出错。

③ 循环冗余校验 (CRC) (数据链路层广泛使用)

原数据 $D \rightarrow k$ 位

预产生 n 位 CRC 校验码, 则事先送 $n+1$ 位 = 进制位模式 G (生成多项式)

例: $D = 1010001101$, $n = 5$

$G = x^5 + x^4 + x^2 + 1$ (110101) ($n+1=6$ 位)

计算产生的校验码 R :

$G \rightarrow 110101 \overline{) 1010001101 00000}$ ← 预产生 R n 位, 补 $n+1$ 0

~~逐位异或~~ →

计算得: $R = 01110$

则实际传输: $1010001101 01110$

101010
 110101
 111110
 110101
 101100
 110101
 110010
 110101
 01110 ← R

补 0 没法除, 只剩 4 位 < 6 位, 将 4 位补为 R 的 5 位

接收方用收到的数据除以 G , 余 0 则正确

校验能力: 少于 $n+1$ 位皆可检错

④ 纠错码

· 海明码 (计组大家都会)



三、基于数据链路层协议

1. 关键假设

分层进程独立假设、(网、数、物进程独立, 进程间通信方式: 传递消息)

提供可靠服务假设 (提供可靠, 面向连接的服务)

只处理通信错误假设 (不考虑断电, 重启等别的问题)

2. 3种协议 (主要是停-等协议的思想)

(1) ① 乌托邦式单工协议 (完美但不现实) (单工协议, 完美信道, 始终就绪, 瞬间完成)

(2) 无错信道上的停等协议

停-等式协议: 发送方发送一帧后暂停, 等待确认帧

接收方完成接收后, 回复确认接收

(3) 有错信道上的单工停等式协议

法1: 解决了帧传输过程中接收方被“淹没”问题来不及处理,
而法2在此解决了通信信道出错, 导致帧出错/丢失的问题

出错: 接收方发送错误确认

丢失: 发送方增设计时器, 超时重传帧

另外, 接收方通过帧的序号确认重复帧并丢弃

接收方接收错误帧时发送上一步成功接收帧的确认

3. 滑动窗口协议

(6) 窗口机制: 发送方和接收方都有一定容量的缓冲区(窗口),

发送方收到确认前可发送多个帧。

序号使用: 循环重复使用有限的帧序号

流量控制: 发送/接收窗口大小记作 W_t/W_r , 表示收到确认前, 可连续发出的最多数据帧数/接收的最多数据帧数
连续



累计确认：不必对收到的分组逐一发送确认，只确认按序到达的最后一个

图示 P84

发送方发帧，窗口下限前移；收确认，上限前移

接收方收帧，窗口下限前移；发确认，上限前移。

11) 后退 N 协议

出错全部重发：收到出错/乱序帧，丢弃所有后续帧，并不发确认

发送端超时后，重传所有未确认帧（出错帧及以后的）

适用场景：接收窗口大小为 1（只能按序接收）

12) ~~选择重传~~ 选择重传协议

只重传收到否认帧的出错帧，或计时器超时的数据帧

适用场景：接收窗口大小大于 1

例：

接收方

1 2 3

1 已接收
按序

2 帧错误，发送错误否认帧，等待重发

3 收到，等待 2 成功收到（序号小的先接收后），再按序交付给上层。



第四章. 介质访问子层

零.

MAC子层在哪?

第三章开头已说明, 介于LLC子层和物理层间

一. 信道分配问题

①

点-点信道: 信道直接连接两个端点.

VS

多点访问: 以以太网拓扑, 早期星型拓扑用集线器, 现在几乎都是交换机

注意: 逻辑拓扑是总线式的, 信道是共享的.

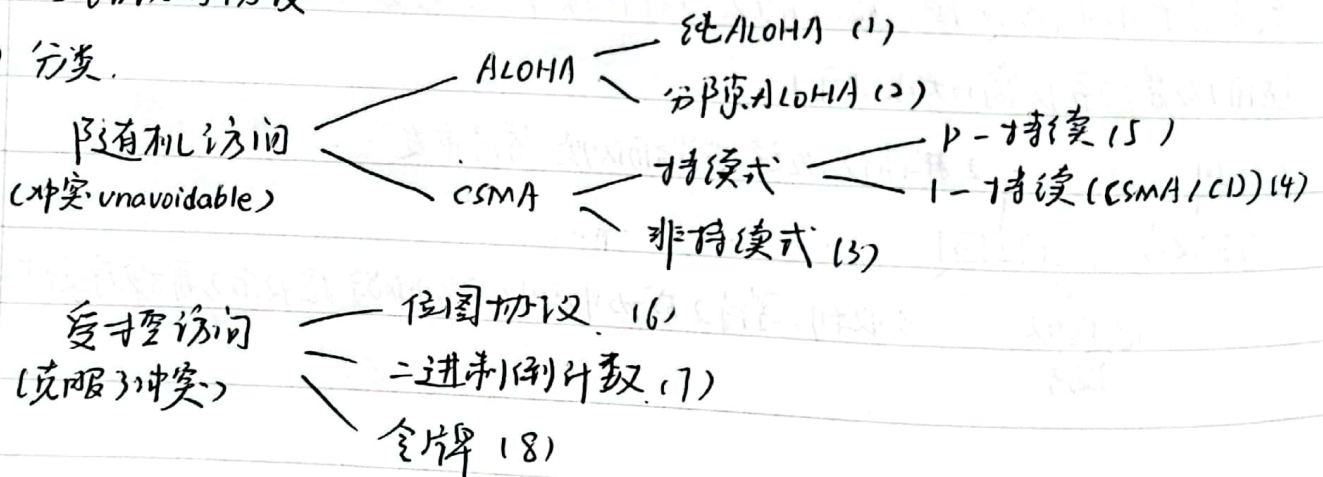
②

常见的局域网拓扑

总线拓扑, 星型拓扑, 环型拓扑, 都共享一根信道

二. 多路访问协议

(1) 分类.



有限竞争协议 — 自适应树 (1)

(利用前两者优势)

(1) 纯ALOHA协议

原理: 想发就发

冲突: 两个或以上的帧, 随时可能冲突.

冲突的帧完全破坏, 需重传, 效率极低



12) 分隙ALOHA协议

把时间分成时隙, 其长度对应一帧的传输时间。

帧的发送须在时隙的起点, 故冲突也只发生在时隙起点。



↓ 优化

13) CSMA: "先听后发"

13) 1 非持续式CSMA

① 倾听, 介质闲则发送, 忙则等待一个随机时间, 重新倾听。

好处: 等随机时间, 减少冲突可能性 坏处: 随机时间内可能介质闲浪费

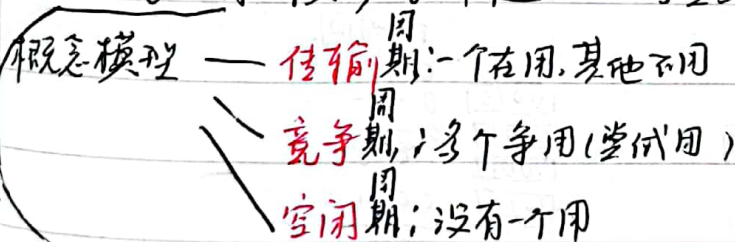
14) CSMA/CD (1-持续) (和网课定义有冲突; 按老师PPT走)

原理: "先听后发, 边发边听"

过程: ① 经倾听, 介质闲, 则发送

② 介质忙, 持续倾听, 一旦空闲立即发送

③ 如果冲突, 等一个随机时间重复①



Why still 冲突?

16 同时发送

(1) 同时发现介质闲)

(2) 传播延迟时间(听到闲的时候实际已经忙了)

15) P-持续式CSMA

→ 翻页



过程: ① 侦听, 介质闲, 则以 p 的概率发送, 以 $1-p$ 的概率推迟一个时间单元发送

② 介质忙, 持续侦听, 一旦闲重复①

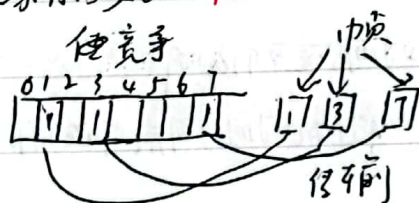
③ 如果发送已推迟一个时间单元, 再重复①

p -持久式: 冲突少, 吞吐量高, 可节省付出了高延迟的代价

16) 位图协议

竞争期 and 传输期

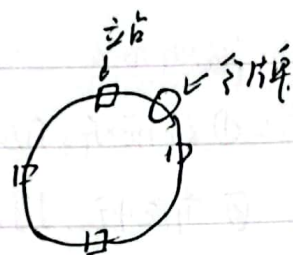
大家有需要 **举手** → 按序发送



17) 令牌传递

令牌: 发送权限

令牌运行: 发送工作站去抓取, 获得发送权
(有令牌的发, 发完释放)



18) 二进制树搜索协议

每个站点: **编号**, 序号长度相同

特点: **高序号站点优先**

	时间			
	0	1	2	3
10010	0	-	-	-
10100	0	-	-	-
10011	1	0	0	-
10101	1	0	1	0
	1	0	1	0

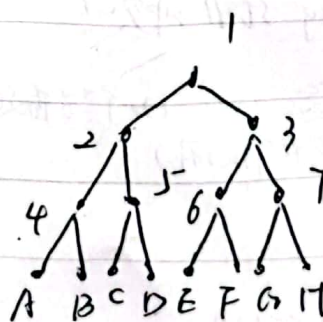
18) 自适应树搜索协议

所有站点同时竞争

只有一站点中消, 则获得信道

否则下一个竞争时隙, 一半站点竞争, 另一半

下一时隙竞争 (递归)



三、五、以太网

1) 经典以太网

- ① max速率 10Mbps
- ② 使用曼彻斯特编码 — 物理层
- ③ 使用同轴电缆、中继器连接
- ④ 主机运行 CSMA/CD 协议
- ⑤ 常用的以太网 MAC 帧格式: — MAC 3 层协议
- ⑥ DIX Ethernet V2 标准 / IEEE 的 802.3 标准
- ⑦ 石更件地址又称物理地址 / MAC 地址, MAC 帧中源/目的地址长度均为 6 字节

帧格式:

目的地址	源地址	类型	数据	校验和
6	6	2	46~1500	4

Ethernet V2 视为上层协议类型
IEEE 802.3 视为数据层协议

12) 交换式以太网

核心: 交换机 (工作在数据链路层, 检查 MAC 帧目的地址对收到的帧转发)

13) 快速以太网 (100Mbps 以太网)

带宽 10Mbps → 100Mbps

原本工作方式保留, 使用双绞线/光纤作线缆
(帧格式, 接口, 过程规则)

14) 千兆以太网 (gigabit Ethernet)

100Mbps → 1000Mbps, 保留工作方式

半双工方式下使用 CSMA/CD (向后兼容), 增加载波扩充, 帧突发

全双工方式不用 CSMA/CD (缺省方式)



15) 万兆以太网 (10-Gigabit Ethernet)

1 Gbps $\rightarrow$ 10 Gbps

只支持全双工, 不再用CSMA/CD

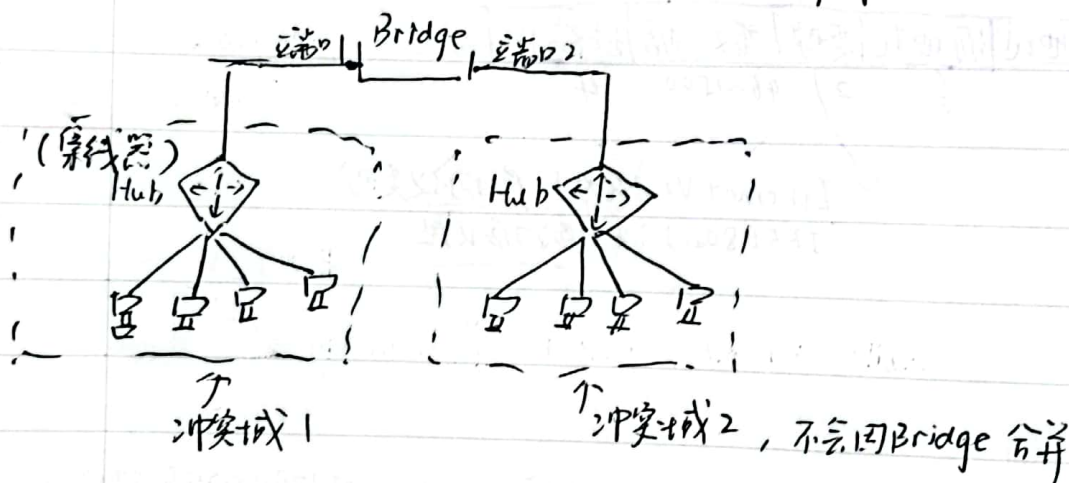
保持兼容性

16) 数据链路层交换原理

(1) 用物理层设备扩充网络: 扩大冲突域



用数~层设备扩充 (如网桥/交换机): 分隔冲突域



(2) 理想的网桥是透明的

- 即插即用

- 网络中的站点无需感知网桥存在 or not

17) MAC地址表的构建

增加表项: 逆向学习 **源地址** (帧的源地址对应项不在表中)

删除表项: 老化时间到期

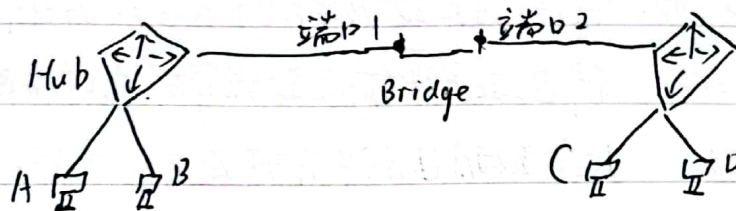
更新表项: 帧的源地址在表中, 更新时间戳



假设经构建后, MAC地址表长这样:

MAC地址表	
MAC地址	端口
MAC-A	1
MAC-B	1
MAC-D	2

利用此表, 介绍网桥利用MAC地址进行数据帧转发
转发的三种情况 (设备网络和网桥如下图)



Forwarding (转发): B → D 发数据帧, 表中 find 了 MAC-D 在不同端口 (2), 网桥转发

Filtering (过滤): B → A 发数据帧, 表中 find 了 MAC-A 在同一端口 (1), 网桥过滤

Flooding (泛洪): B → C 发数据帧, 无 MAC-C (端口匹配失败, 从所有端口发出, 每个桥除了入口)

泛洪: 两种目的地址的帧需泛洪

广播帧: 目的地址为 FF-FF-FF-FF-FF-FF 的数据帧

未知单播帧: 目的地址不在 MAC 地址表中的单播数据帧



第五章 网络层

一、网络层服务

(1) 网络层实现端系统间多跳传输可达

发送端：将传输层数据单元封装在数据包中

接收端：解包，取出传输层数据单元，交付给传输层

路由器：检查数据包首部并转发

(2) 关键功能

① 路由：选择数据包从源端到目的端的路径。（路由算法与协议）

② 转发：将数据包从路由器的输入接口传送到正确的输出接口

(3) 无连接服务的实现

• 网络层向上只提供简单灵活的无连接的尽最大努力交付的数据包服务
(不提供服务质量的承诺)

• 尽力而为交付

① 不提供端到端的可靠传输服务：丢包、乱序、错误

优点：网络的造价↓，运行方式灵活，能够适应多种应用

二、Internet网际协议

(1) IPv4协议 (★)

①

版本		首部长度		区分服务		总长度	
标志		生存时间		协议		片偏移	
源地址		目的地址		首部校验和		选项 (长度可变)	
数据部分						填充	



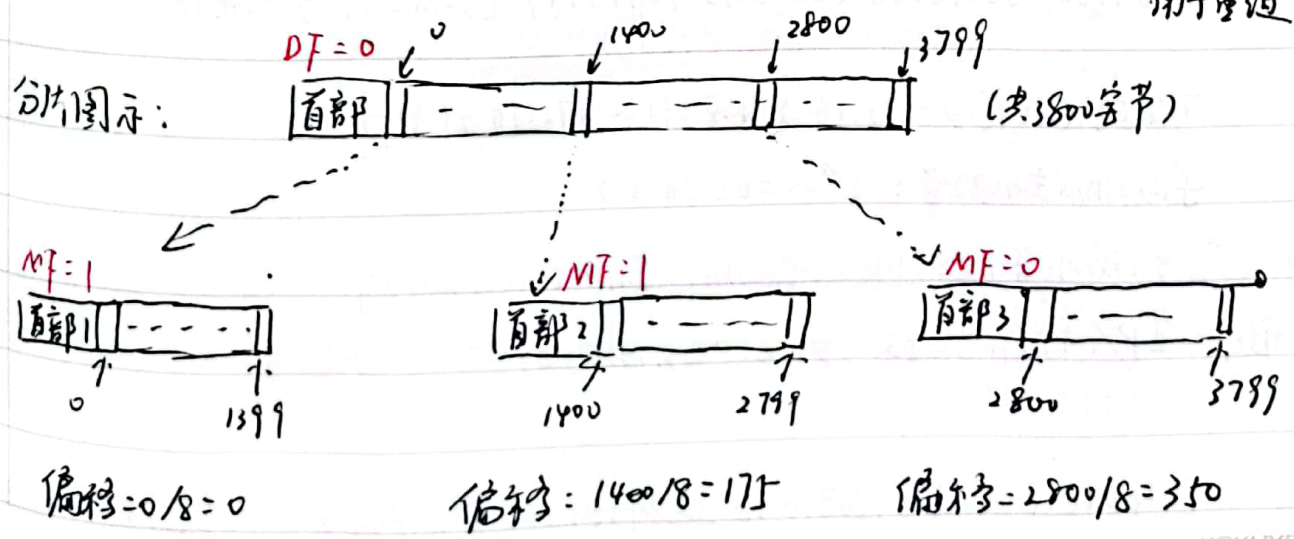
- 版本: 4 bit, 表示采用的IP协议版本
- 首部长度: 4 bit, 整个IP数据包头部的长度(除数据)
- 区分服务: 8 bit, 略
- 总长度: 16 bit, 整个IP数据报文长度 $\text{Max} = 2^{16} - 1 = 65536$ 字节
- 标识: 16 bit, 用以标识同一源数据包的碎片
- 标志: 3 bit, 目前只有两位有意义: MF (more fragment), DF (don't fragment)
- 片偏移: 13 bit, IP分片后, 该IP片在总IP片的相对位置 (单位为8字节)
- 生存时间TTL: 8 bit, 表示数据包在网络中的生命周期, 每经过一个路由器减1
- 协议: 8 bit, 标识上层协议 (TCP/UDP/ICMP)...
- 首部校验和: 16 bit, 对数据包首部的校验, 不含数据部分
- 源地址: 32 bit, 源IP地址
- 目的地址: 32 bit, 目的IP地址
- 选项: 可扩充部分, 长度可变, 定义了安全性, 严格源路由, 记录路由等选项
- 填充: 用全0的填充字段补齐为4字节(32 bit)整数倍

② 数据包分片

概念: MTU: 最大传输单元

允许途中分片: 根据下一跳链路的MTU分片

重组策略: 互联网采用目的端重组 (IPv4也是), 标识, 标志, 片偏移用于重组



2) IP地址

① **IP地址**: 网络上的每台主机(路由器)的每个接口都分配全球唯一的32位标识符。网络号相同的这块连续IP地址空间称为地址的**前缀(网络前缀)**

② IP地址分类

点分十进制(每段0-255, 对应8位二进制, 共32位)

1.0.0.0 ~ 127.255.255.255	A类	0 - - - -
		128
128.0.0.0 ~ 191.255.255.255	B类	10 - - -
		128+64
192.0.0.0 ~ 223.255.255.255	C类	110 - - -
		128+64+32
	D类	1110 - - -

IP特殊地址: PPT P156

③ 子网划分

在网络内部将网络进行划分以供多个内部网络使用, 对外仍是一个网络

子网掩码: 32bit二进制数, 置1为网络位, 置0为主机位

△: 通过三级IP地址(网络号+子网号+主机号)和子网掩码计算网络地址

172	16	2	160
255	255	255	192
10101100 00010000 00000010 10100000			
11111111 11111111 11111111 11000000			
10101100 00010000 00000010 10000000			
10101100 00010000 00000010 10111111			

前缀 26位

主机位 6位

(主机位全0, 子网地址)

(主机位全1, 广播地址)

可分配地址范围: 172.16.2.(128+1) ~ 172.16.2(191-1)

子网拥有主机数量: $2^n - 2 = 62$ ($n=6$)

④ 无类域间路由(CIDR) (Classless Inter-Domain Routing)

形式: IP/网络前缀位数, 如 200.23.16.0/21

(斜线记法)

一个CIDR地址块可表示很多地址, 这种地址的聚合常称为**路由聚合/构成超网**



最长前缀匹配

例: IP地址 132.19.232.5, 应被转发到哪一个下一跳路由器

A: R₁ 132.0.0.0/8

B: R₂ 132.0.0.0/11

C: R₃ 132.19.232.0/22

D: R₄ 0.0.0.0/0

先看 A, 132 匹配 8 位

0 → 00000000

看 B, 132 匹配 8 位 19 → 00010011, 匹配 3 位, 网络 31 位 > 8 位。

232 → 11101101

看 C, 132.19 匹配 16 位 232 → 11101000 匹配 5 位, 共 21 位 < 22, 舍弃。

选 B

⑤ IP包转发

(0) 间接交付: 与目的主机不在同一 IP 子网内

首部

IP 与 MAC 地址: IP 地址放在 IP 数据包首部, MAC 地址 (硬件地址) 放在 MAC 帧

IP 数据包经过不同链路时, IP 地址不变, 而 MAC 帧中硬件地址改变

网课《计算机网络》(王道) p50 讲得清楚些

(1) ARP 协议

ARP 表中缓存了主机的 IP 地址和 MAC 地址映射关系 (每个主机有一 ARP 表)

设主机 A 已知 B 的 IP 地址, 欲获得其 MAC 地址

Case 1: 表中有, 直接得

Case 2: 表中无, 广播发送 ARP 请求分组, B 在同一子网内, 则 B 向 A 单播

发送 ARP 响应分组, A 在 ARP 表中缓存 B 的 IP 和 MAC 地址映射关系,

超时删除

Case 3: 表中无, B 不在同一子网内, 经路由器多跳, 多次 ARP 协议直到跳到有 B 的子网内 (路由到另一局域网)



⑥ 网络地址转换 (NAT) Network address translation

用于解决 IPv4 地址短缺问题。化私有(保留)地址为公有 IP 地址

↓ A.B.C 类地址

例 P169

~~理解~~ 我的理解: 通过 NAT 转换表将所有从本地网络发出的报文伪装以相同
的单-源 IP 地址, 端口也转换, 收到以后转换回去, 去掉伪装
标签

PPT 给出的工作机制:

出数据包: 用 NAT IP 地址(全局), 新 port # 替代源 IP 地址(私有), port #
NAT 转换表 每个 (源 IP 地址, port #) 到 (NAT IP 地址, 新 port #) 映射
入数据包 用 NAT 表换回来 ② → ①

⑦ IPv4 地址如何获取

10) 公有 IP 地址全球唯一 -

• 静态设置: 申请固定 IP 地址, 手工设定, 如路由器、服务器

动态获取: 当主机加入 IP 网络: 允许主机从 DHCP 服务器动态获取 IP 地址

11) DHCP (动态主机配置协议) Dynamic Host Configuration Protocol

工作模式: 客户端/服务器模式 (C/S), 基于 UDP 工作

工作过程: ① DHCP 客户端从 UDP 端口 68 以广播形式向服务器发送发现报文 (DHCPDISCOVER)

② DHCP 服务器以广播形式发送提供报文 (DHCPOFFER)

③ 客户端从多个 DHCP 服务器中选取一个, 以广播形式发送请求报文 (DHCPREQUEST)

④ 被选中的 DHCP 服务器以广播形式发送确认报文 (DHCPACK)

(DHCP 服务器不仅返回客户端所需 IP 地址, 还有缺省路由器 IP 地址, DNS 服务器 IP 地址, 网络掩码)



类型3, 时间超时类型11

⑧ ICMP ICMP+协议 (互联网控制报文协议)

分类: ICMP差错报告报文: 终点不可达: 不可达主机, 不可达网络, 无效端口, 协议

ICMP询问报文: 回送请求/回答 (ping使用)

Ping & ICMP

用以测试两主机间的连通性 (显示往返时延 ms, 生存时间 TTL)

Traceroute & ICMP

- 获取整个路径上的路由器地址
- 从源向目的发送一系列UDP段 (不可能的端口号), TTL=1, TTL=2...
- 当第n个遇到第n个路由器: 丢弃包, 向源发送一个ICMP报文 (类型11, 时间超时)
- 源收到ICMP报文, 计算RTT (Traceroute + 对同一RTT值执行三次上述过程)
- 停止条件: UDP段到达目的主机, 目的返回ICMP (类型3, 端口不可达) 分组, 停止

三. 路由算法

0. 须满足特性

正确性 / 简单性 / 鲁棒性 / 稳定性 / 公平性
(健壮, 强性, Robust)

静态路由 / 动态路由选择策略

1. 三种路由算法; 两种路由算法

- ① 最短路径算法 (Dijkstra) 全局更新, 一个Dijkstra算完所有
 - ② 距离向量路由 (Bellman-Ford 算法) 邻居交换信息, 更新, 迭代, 分布式 <王道> P54
 - ③ 链路状态路由 (Link-State): 思想类 Dijkstra
1. 发现邻居, 了解他们的网络地址
 2. 设置到每个邻居的成本度量
 3. 构造一组, 分组中包含自己收到的所有信息



扫描全能王 创建

4. 将此分组发送到其他的路由器.

5. 计算到其他路由器的最短路径.

例: P199

④ ② DV vs ③ LS

(1) DV: 邻居间交换, LS: 全网扩散

DV: 部分通听途说 LS: 自己测量

DV: 计算结果传递, 健壮性差 LS: 各自计算, 健壮性好

DV: 收敛又慢, 可能计算到无穷(16) LS: 收敛快

四 Internet 路由协议

1. OSPF (开放式最短路径优先协议) Open Shortest Path First, 使用 IP

(0) 采用分布式的链路状态计算

(1) 基本思想:

① 向本自治系统内所有路由器洪泛信息

② 发送的信息为本路由器相邻的所有路由器的链路状态

③ 只有链路状态发生变化时路由器才洪泛再次发信息

(2) 链路状态:

指: 相邻路由器, 及度量: 费用, 距离, 时延, 带宽等

• 由于各路由器间频繁交换链路状态信息, 因此所有路由器最终都能建立一个链路状态数据库 LSDB, 实际就是区域内的拓扑结构图 (区域内一致)

(3) 区域

使用层次结构的区域划分 (上层的叫主干区域), 以减小 LSDB 规模

路由器因此分为: 内部路由器 / 区域边界路由器 / 自治系统边界路由器

IR
(Inner)

ABR
(Area Brouder)

ASBR
(AS Brouder)

注: 如有计算题, 按 Dijkstra 走



2. RIP (路由选择协议) Routing Information Protocol, 使用IP

(1) 基于距离向量路由算法

(1) 使用跳数衡量到达目的网络距离, 跳数少 $\rightarrow$ 距离短
(一条路径最多只能包含15个路由器) $\rightarrow$ 故适用中小型网络

(2) 基本原理

① 仅和相邻路由器交换信息

② 交换内容为自己的路由表

③ 周期性更新: 30s

(3) 工作过程, <<王道>> P54, PPT没详细讲规则

④

3. BGP (边界网关协议) Border Gateway Protocol, 使用TCP

(1) 区别: OSPF, RIP 还有 ISIS 等是 内部网关协议 IGP

BGP 是 外部网关协议 EGP

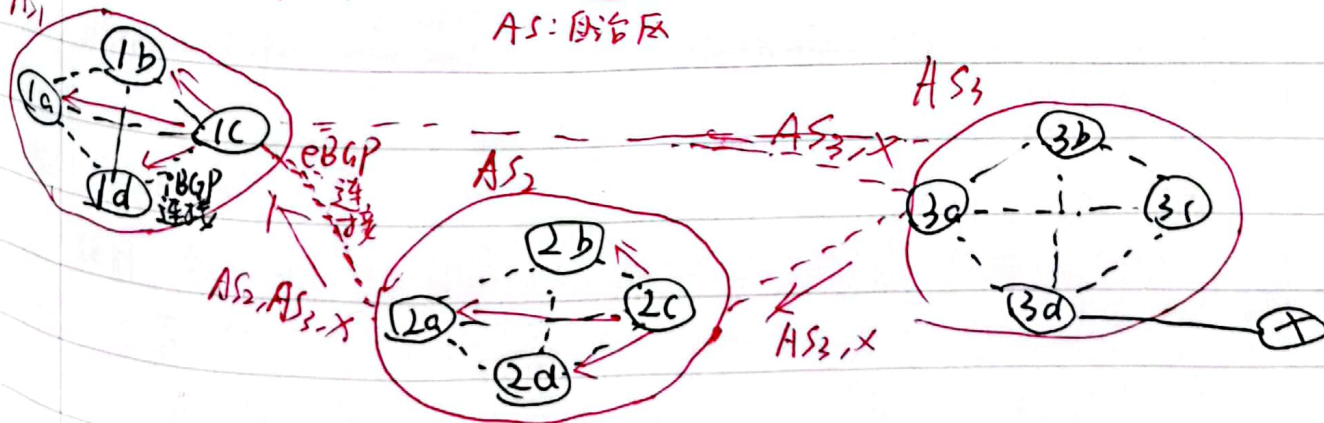
(且BGP是目前互联网唯一实际运行的自治域间的路由协议)

(1) 功能: eBGP: 从相邻的AS获得网络可达信息

iBGP: 将网络可达信息传播给AS内的路由器

external $\rightarrow$ inner (第(3)点有图)

(2) BGP会话: 两个BGP路由器通过TCP连接交换BGP报文
AS: 自治区



AS1 路由器 1c 可能从 2a 学到 AS2, AS3, X

从 3a 学到 AS3, X

于是, 可选择 AS3, X, 并在 AS1 中通过 iBGP 通告给



扫描全能王 创建

(3) BGP特点:

- BGP协议交换路由信息的节点数量级是自治系统数的量级
- BGP运行时, BGP的邻居交换整个BGP路由表, 以后只需更新所有变化部分
- 根据“可达信息”和“策略”决定路由。
 ↓
 eBGP提供 IBGP决定



第六章 传输层

一. 传输层复用和分用

10) 传输层基本服务

将主机间交付扩展到进程间交付, 通过复用和分用实现

发送端复用: 传输层从多个套接字收集数据, 交给网络层发送

接收端分用: 传输层将从网络层收到的数据, 交付给正确的套接字
(每个套接字内有唯一标识)

11) 套接字端口号的分配

① 端口号 (16 位整数) 是套接字一部分, 每个套接字在本地主机一端一个。

② 套接字端口号分配: 自动分配 / 使用指定端口号创建套接字
通常客户端 通常服务器端

③ UDP 分用方法:

UDP 套接字使用 < IP 地址, 端口号 > 二进制标识

UDP 编程:

```
DatagramSocket mySocket2 = new DatagramSocket(9157);
```

源端口号: source port: 6428 9157
目的端口号: dest port: 9157 6428

```
DatagramSocket serverSocket = new DatagramSocket(6428);
```

④ TCP 分用方法

使用 < 源 IP 地址, 目的 IP 地址, 源端口号, 目的端口号 > 四元组标识连接套接字

一个监听套接字

and

多个连接套接字

只接收报文段的
连接字目的端口号
= 服务器监听套接字的端口号

等待连接请求,
端口号众所周知

收到连接请求后创建一个, 使用临时分配的端口号
每个连接套接字只有一个客户通信



二、^连无连接传输: UDP

(1) UDP提供的服务:

进程到进程间的报文交付; 报文完整性检查(可选): 检错则丢弃

(2) UDP需要实现的功能:

复用和分用 (报文中16位源端口号, 16位目的端口号)

报文检错 (报文总长度, 校验和)

(3) 利用UDP校验和检错

举例: P233

计算UDP校验和时, 包括(伪头, UDP头, 数据三部分)

伪头: 一个虚拟数据结构, 取自IP报头, 并非UDP报文实际有效成分

(4) 为什么需要UDP?

应用可尽快发送报文, 无建立连接的延迟, 不限制发送速率
报文开销小, 协议处理easy

(5) UDP适合哪些应用

容忍丢包但对延迟敏感的应用: 如流媒体.

以单次请求/响应的应用: 如DNS

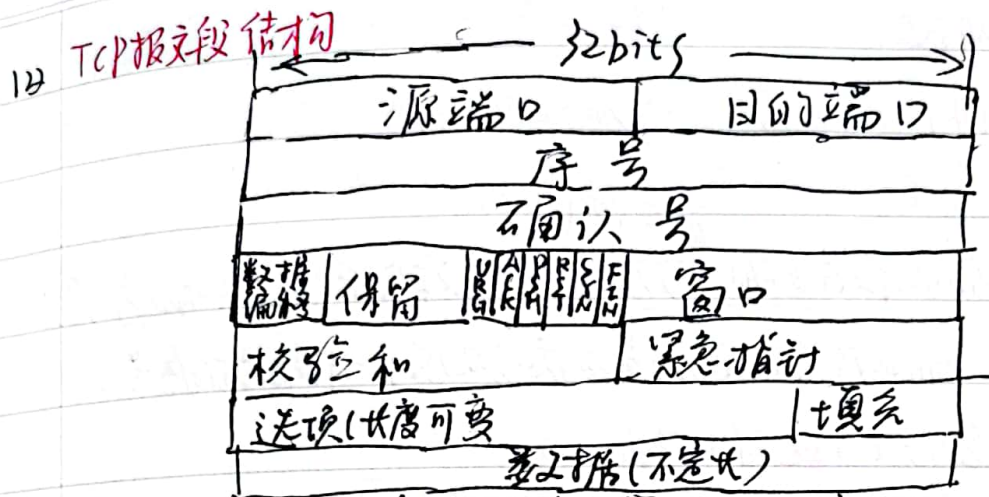
若应用要求基于UDP进行可靠传输: 应用层实现可靠性

三 TCP

(1) 概述部分

TCP服务模型: 在一对通信的进程间提供一条理想的字节流管道
点对点通信, 全双工, 可靠, 有序的字节流





序号 — 发送序号: 数据载荷中第一个字节在字节流中的序号

确认号 — 确认序号: 期望接收的下一个字节的序号。

数据偏移 (TCP头长度): 以4个字节为单位, for example: 1111 → 60个字节

URG: 紧急标志, 需先发可插入
(Acknowledge)

ACK=1: 确认序号有效 (在连接建立后, 所有发送的报文段必须把Ack置1)

PSH=1: 接收方尽快交付 (URG=1时常带), 不用等前面的交付

RST: ~~置1~~ = 1: 连接出错, 需释放重建

SYN=1: 表明此报文为一个连接请求/连接接受报文 (前两次握手为1, 后为0)

FIN=1: 表明此方发送方完成传输 (双方可一个先完成, 一个再传完后再完成)

窗口: 接收端还可接收的字节数

选项: 最大段长度 MSS: TCP段中可携带的最大数据字节数

选择确认 SACK: 改进的TCP引入选择确认, 可指出缺失的数据字节
接收端

→ **初始序号的选取**: TCP实体维护一个32位计数器, 每4微秒增1

13 TCP可靠数据传输

10) TCP在不可靠的IP服务上建立可靠的数据传输

11) 机制: 发送端: 流水线式发送数据, 等待确认, 超时重传

接收端: 差错检测, 采用累积确认机制



12) 一个高度简化的TCP协议.

- 仅考虑可靠传输机制, 且数据仅在一个方向上传输

① 接收方:

→ 类似回退N

确认方式: 采用累积确认, 仅在正确按序收到报文段后, 更新确认序号;
其他情况, 重复前一次的确认序号 (以确认序号标志的是下一个期待的序号)
失序报文段处理: 先缓存 (与选择重传类似)

② 发送方:

流水线式发送报文段.

→ 发送该报文段后重启

定时器的使用: 仅对最早未确认的报文段使用一个重传定时器 (类似回退N)

重发策略: 仅在超时时重发最早未确认报文段 (类似选择重传, 有缓存不用重发)
收到ACK: 如果确认序号大于已发送未确认的最小序号, 推进发送窗口 → 更新
如果发送窗口中还有未确认报文段, 启动定时器

③ 接收端的事件和处理

- 为减少通信量, TCP允许接收端推迟确认
于是, 有以下几种情况 (事件):

1. 收到一个期待的报文段, 且之前报文段均已发送过确认
动作: 推迟发送确认, 在500ms时间内无下一个报文段到达, 发送确认

2. 收到一个期待的报文段, 且前一个报文段被推迟
动作: 立即发送确认 (估计RTT的需要)

3. 收到一个失序的报文段, 检测到序号间隙
动作: 立即发送确认 (快速重传的需要), 重复当前确认号.

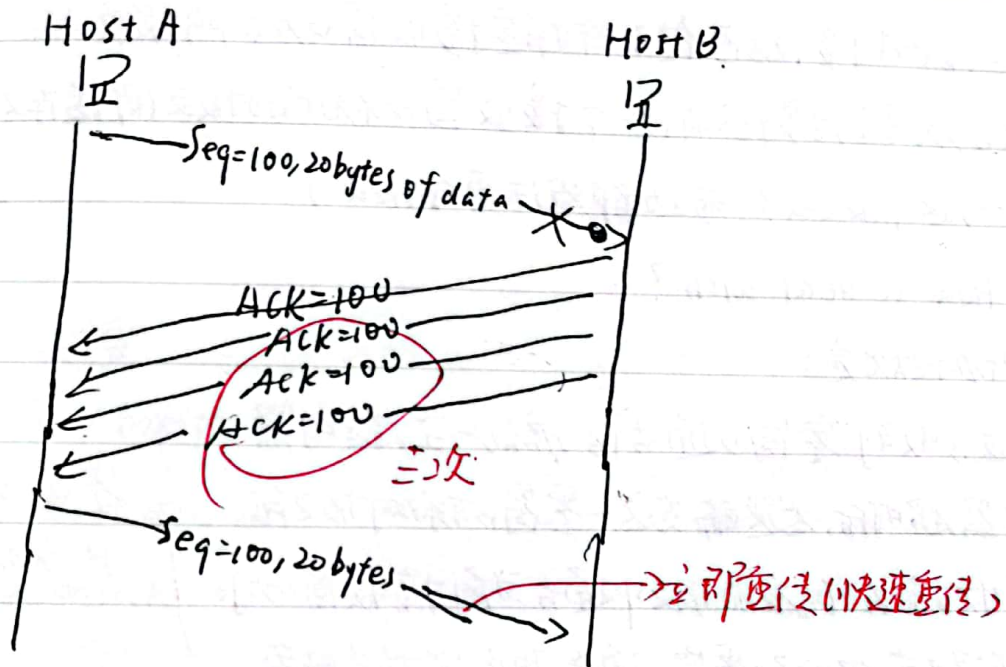
4. 收到部分或全部填充间隙的报文段.
动作: 若报文段始于间隙的开端, 立即发送确认 (推进发送窗口), 更新确认序号



④ 快速重传。(Why? 一仅靠超时重传, 恢复太慢!)

发送方可利用重复ACK检测报文段丢失

TCP协议规定:当发送方收到对同一序号的3次重复确认时(加上开始一共4次)立即重发包含该序号的报文



⑤ 可靠传输 通过这一系列规则,大量减少因Ack丢失,定时器过早超时而引起的重传

(4) TCP流量控制

- Why?

进入接收缓存的数据未必被立即取走,取完,太快→缓存溢出

Why 回退 N, 送掉重传. UDP not?

① 回退人体, 选择重传假设正确. 按序到达的分组立即交付给上层.

② ODP不保证交付: 缓存溢出, 不归我管!

• How?

① 接收窗口 (接收缓存可用空间)

接收方将其置于报头，通常发送方



发送方则限制已发送,未确认的字节数不超过接收窗口大小。

② 接收窗口为0情况.

- 发送方必须停止发送

注意,此时接收方已传了一个指定接收窗口为0的报文

那么,须等到接收方再传一个接收窗口不为0的报文(即缓存又有空间).

可万一,这个报文丢了,两边都没法再传报文了.

So, How to deal with?

TCP协议规定:

发送方收到“零窗口通告”后,启动一个定时器.

定时器超时后,发送端发送一个窗口探测报文段

接收端在响应的报文中通告当前接收窗口大小.

若发送端仍收到零窗口通告,重新启动定时器.

“非零窗口通告”

零窗口探测的实现

③ 粘滞窗口综合症

What?

数据发送快,接收慢,消费慢,零窗口探测的主要问题:

接收方不断发送微小窗口通告;发送方不断发微小分组 → 带宽浪费.

How to deal?

I. 接收方启发式策略.

仅当窗口大小显著增加(达缓存空间一半或一个MSS,取两者min)后,才发送更新的窗口通告

II. 发送方启发式策略

聚集足够多的数据才发

- Nagle算法



- ① 在新建连接上, 应用数据到来时, 组成一个TCP段发送(无论多小)
 - ② 收到确认以前, 后便应用数据放入发送缓存
 - ③ 数据量达一个MSS或上一次传输的确认到来 (取两者较小时间), 用一个TCP段发完全部缓存.
- 算法优点: 适应网络延时, MSS长度及多用速度各种组合/不止吞吐量

1.1 TCP连接管理

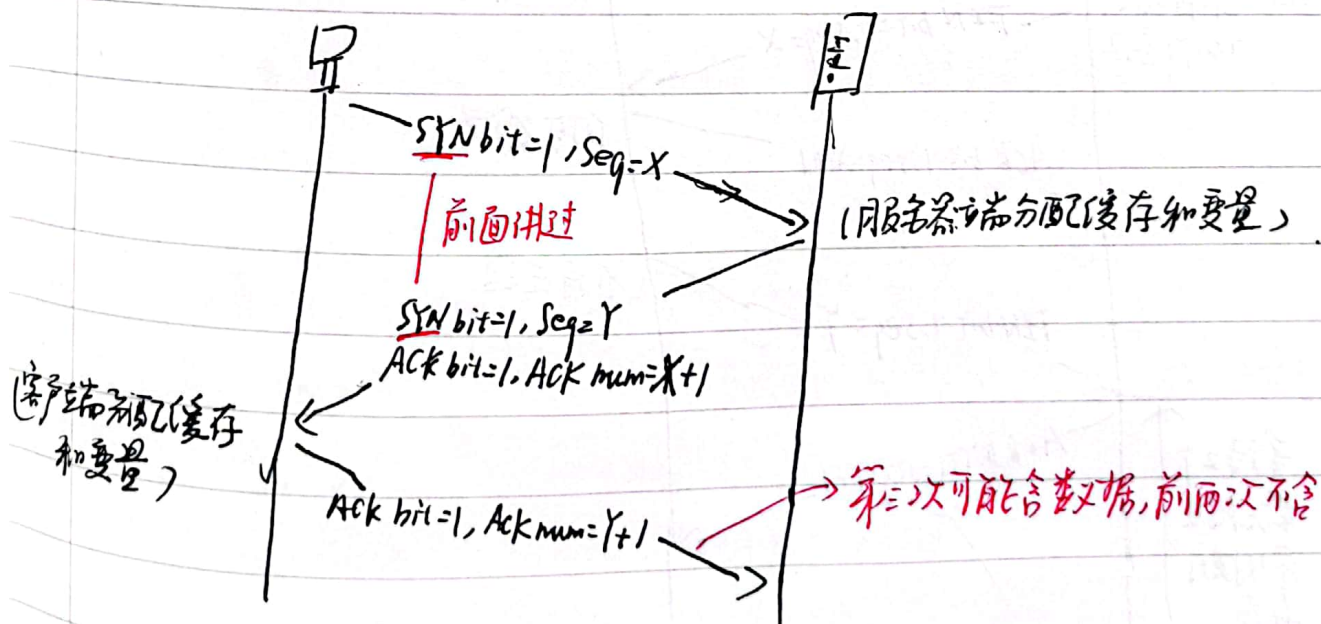
① 建立一个TCP连接

- 确认双方同意 / 初始化参数(序号, MSS等)

```
Socket clientSocket = new Socket("hostname", "port number");
```

```
Socket connectionSocket = welcomeSocket.accept();
```

② 三次握手建立连接



```
客户: clientSocket = socket(AF_INET, SOCK_STREAM);  
LISTEN;
```

```
clientSocket.connect((serverName, serverPort));
```



服务器:

```
serverSocket = socket(AF_INET, SOCK_STREAM)
```

```
serverSocket.bind(('', serverPort))
```

```
serverSocket.listen(1)
```

```
connectionSocket, addr = serverSocket.accept()
```

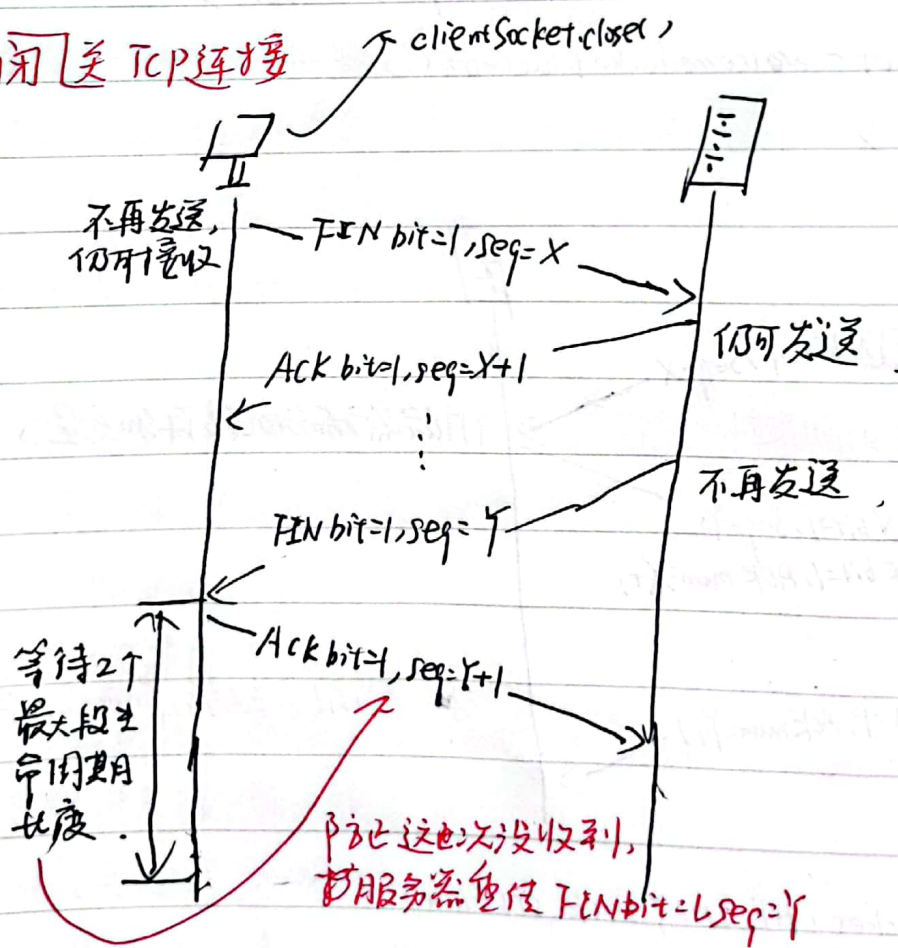
起始序号选择 (前未提及)

- 必须避免新旧连接上的序号产生重叠。

SO - ΔT 取 4 微秒 (较小值), 确保发送序号的增长速度比起始序号的慢

使用 32 位字节序号 (较长): 确保序号回绕时间远大于分组在网络中最长寿命

② 关闭 TCP 连接



⑥ TCP 拥塞控制

⑥ compared to 流量控制: 拥塞控制是发送方感知拥塞后, 限制发送速率
而流量控制是接收方因缓存不够而做出的限制

① 发送方感知拥塞, 限制发送速率机制

· 频繁丢包, 分组延迟增大 (拥塞后果)

↳ 重传定时器超时 / 发送端收到 3 个重复 ACK

机制: 拥塞窗口 cwnd 限制已发送未确认的数据量。

$$\text{rate} = \frac{\text{cwnd}}{\text{RTT}} \text{ Bytes/sec}$$

② 拥塞窗口的调节策略: AIMD

I. 乘性减 (Multiplicative Decrease)

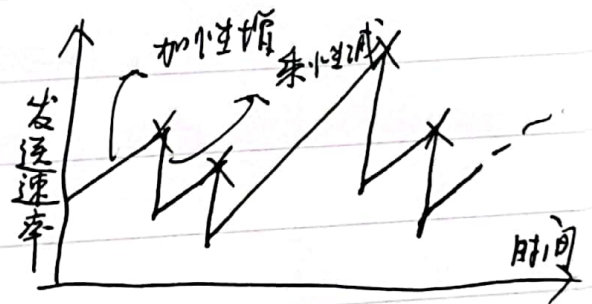
发送方检测到丢包后, 将 cwnd 大小减半

(迅速减小发送速率, 缓解拥塞)

II. 加性增 (Additive Increase)

若无丢包, 每经过一个 RTT, 将 cwnd 增大一个 MSS, 直到检测到丢包。

(缓慢增大, 迟滞振荡)



④ 优化: 小慢启动 (只是相对于早期策略慢, 实际快)

Why 优化: 加性增, 从 cwnd = 1 MSS 慢慢加, 带宽可能很大, 增速太慢!

优化: 慢启动: 新建连接时指数增大 cwnd, 直至丢包

具体实施: 每收到 1-TACK 段, cwnd 增加 1-MSS

收 2 增 2, 收 4 增 4, 相当于每次翻倍

⑤ 丢包事件



⑤ 丢包重传事件

丢包 - 超时 —— 网络交付能力很差

收到3个重复ACK —— 网络仍有一定交付力

ssthresh: 慢启动转为拥塞避免的阈值 (slow start threshold)

收到3个重复ACK, 设置 $ssthresh = cwnd/2$, $cwnd = ssthresh + 3$, 并加倍

超时: 设置 $ssthresh = cwnd/2$, $cwnd = 1MSS$, 并慢启动

例图: P282



第七章 应用层

一、应用层概述

- ① 每个应用层协议都是为了解决某应用问题，通过位于不同主机中的多个应用进程之间的通信和协同工作完成。应用层的具体内容就是规定应用进程通信时遵循的协议。

① 应用进程

为解决具体应用问题而彼此通信的进程

二、应用进程通信方式

1) C/S (客户/服务器) 方式

- ① 客户/服务器指通信中涉及的两个应用进程

· 客户/服务器方式描述应用进程间不提供服务和服务的关系。

· C/S方式可无连接，也可面向连接（此时，C/S通信关系一旦建立，通信就是双向的）

② 客户进程特点

主动向远地服务器发起通信

必须知道服务器进程所在主机的IP地址才能发出服务请求

① 服务器进程特点

必须处于运行状态才能提供服务（通常系统启动时自动调用并一直运行）

被动等待并接受多个客户通信请求

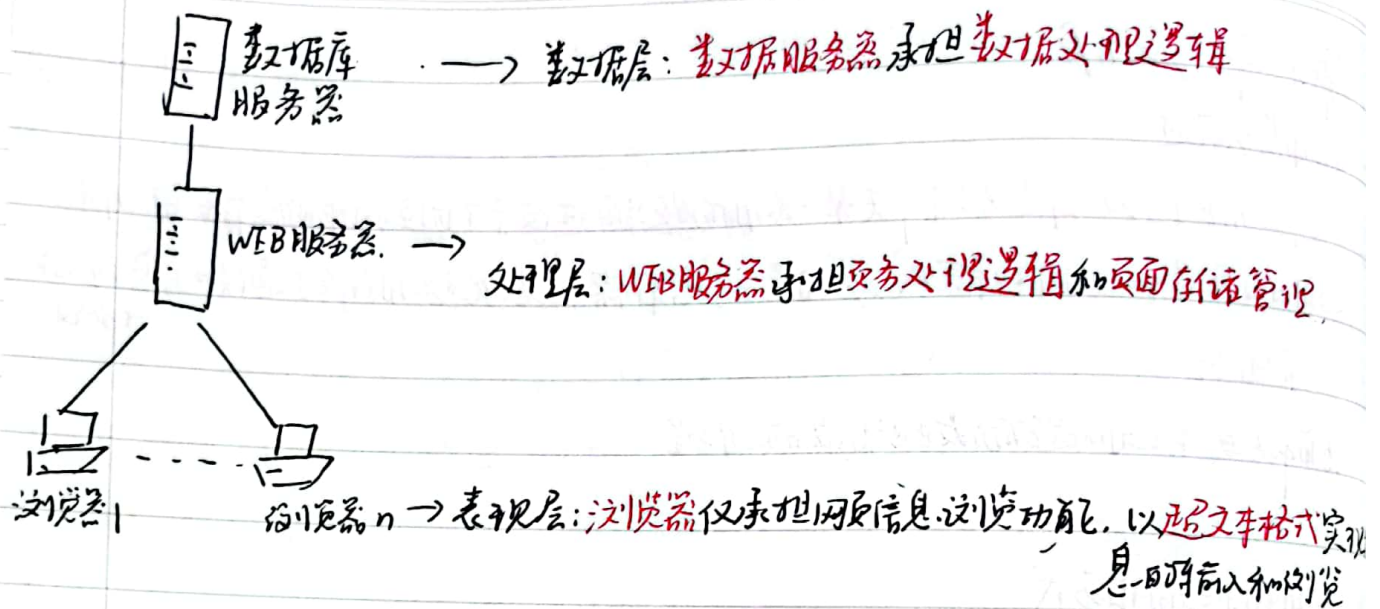
2) B/S (浏览器/服务器) 方式 (C/S特例)

- ① B/S采取浏览器请求/服务器响应的交互模式

B/S方式下，用户界面完全通过Web浏览器实现，一部分事务逻辑前端实现，主要事务逻辑在服务器端实现

② B/S的3层架构





③ B/S特点

界面统一、使用简单

易于维护, 可扩展性好, 信息共享度高

↓ HTML是数据格式的一个开放标准

1.3) P2P(对等)方式 Peer to Peer

① 对等方式指两个进程通信时不存在服务请求方提供方, 平等, 对等的通信
每个P2P进程既是客户, 同时也是服务器

② 工作方式:

I 循环方式 (无连接的服务)

一次只运行一个服务进程, 按多客户进程请求的前后顺序依次响应(阻塞式)

II 并发方式 (面向连接的TCP服务)

· 可同时运行多个服务进程, 每个都对某特定的客户进程作响应(非阻塞式)

· TCP服务进程与多个客户进程之间建立多条TCP连接
一个TCP连接对应一个(熟知)服务端

主服务进程在熟知端口等客户进程请求, 一旦收到, 创建一个从属服务进程



并指明从属服务进程使用临时套接字与客户建立连接 (P302 号, 303)

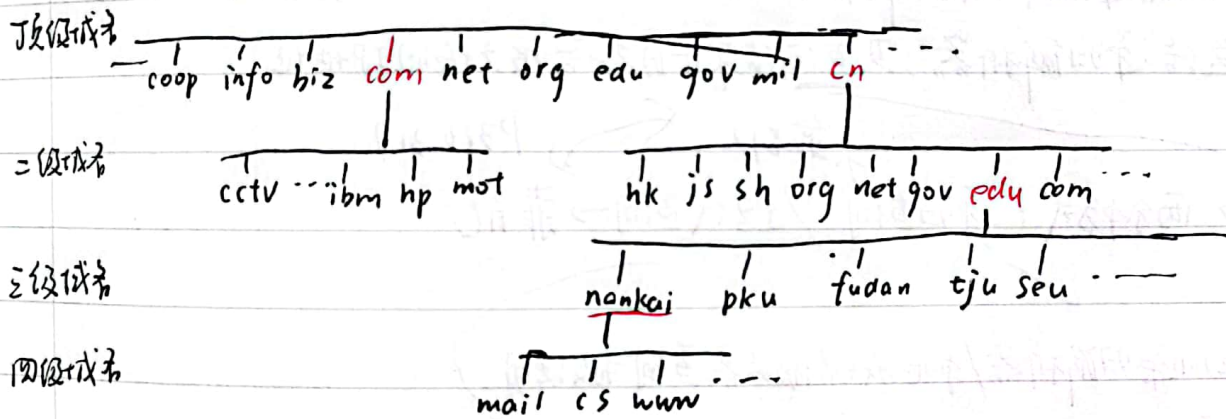
三. 域名系统

① 历史与概述

- ① 域名系统 (DNS, Domain Name System) 是向所有需要域名解析的应用提供服务。
主要负责将可读性好的域名映射成 IP 地址 (如用 `www.baidu.com` 代替 `100.100.100.100`)
- ② Internet 采用层次结构的命名树作为主机名, 并使用分布式的域名系统 DNS



树根



比如: `mail.nankai.edu.cn` 根 层次树状结构的命名方法

四级 三级 二级 顶级

② 顶级域名分类

国家或地区顶级域 TLD (ccTLD): 如 .cn 中国, .us 美国, 目前 200 多个

基础设施域 .arpa, 专用于 Internet 基础设施目的

通用顶级域 gTLD, 早期 20 个, 现已 1200 多个

④ 域名服务器

1. 保存关于域树的结构和设置信息的服务器程序, 域名解析过程对用户透明



扫描全能王 创建



2. SMTP协议 (simple mail transfer protocol), TCP/IP协议上

利用TCP可靠地从客户向服务器传递邮件。使用端口25

直接投递: 发送端到接收端

三个阶段: 连接建立, 邮件发送, 连接关闭

3. POP3协议 (TCP/IP上)

当用户代理打开一个到端口110上的TCP连接后, 客户/服务器开始工作

↓
C/S工作方式

三个阶段 — 认证阶段: 客户输入用户名密码, 服务器响应

事务处理阶段: list, retr, dele 邮件, 用户收邮件, 标记为删除

更新阶段: 将标记删除的邮件删除

4. IMAP协议 (Internet Mail Access Protocol), 运行在TCP/IP协议上

- 用于最终交付的重要协议, POP3改进版, 端口143
- IMAP服务器把每个邮件与文件夹联系起来
- 允许用户代理获取邮件某些部分 (如只获取头部)

五 其他应用层协议

1. Telnet (远程登录) - C/S方式, 使用TCP连接通信 (23端口)

引入NVT (网络虚拟终端) Network Virtual Terminal

NVT: Telnet协议定义的通用字符集

2. FTP (文件传输协议) - C/S方式, 使用UDP协议 (TCP 21端口)
File Transfer Protocol

3. SNMP (简单网络管理协议) - C/S方式, 使用UDP协议
(Simple Network Management Protocol) 目前版本: SNMPv3

